

Transformer Architectures and Variants

A Mathematical and Architectural Treatment

Part 1

Foundations and the Original Transformer

Contents

1	Transformer Architecture: Foundations	1
1.1	The Basic Problem	1
1.2	Sequence Representation	2
1.3	Why Positional Information Is Required	2
1.4	Sinusoidal Positional Encoding	2
2	The Original Transformer	4
2.1	Overall Structure	4
2.2	Encoder	4
3	Self-Attention	6
3.1	Input to the Attention Layer	6
3.2	Queries, Keys and Values	7
3.3	Query–Key Interaction	7
3.4	Scaling	8
3.5	Softmax	8
3.6	Value Aggregation	9
3.7	Complete Scaled Dot-Product Attention	9
4	Multi-Head Attention	10
4.1	Motivation	10
4.2	Attention in Each Head	10
4.3	Concatenation	10
4.4	Dimensions	11
5	Residual Connections and Normalization	12
5.1	Residual Connection	12
5.2	Layer Normalization	12
5.3	Attention Sublayer	13
6	Position-wise Feed-Forward Network	14
6.1	Input	14
6.2	First Linear Transformation	14
6.3	Nonlinear Transformation	14
6.4	Second Linear Transformation	15
6.5	Application to the Complete Sequence	15

7 Complete Encoder Layer	16
8 The Transformer Decoder	17
8.1 Purpose of the Decoder	17
8.2 Decoder Input	17
8.3 Causal Mask	17
8.4 Decoder Self-Attention	18
8.5 Encoder–Decoder Cross-Attention	18
8.6 Decoder Layer	19
9 Output Generation	20
9.1 Decoder Representation	20
9.2 Vocabulary Probability Distribution	20
9.3 Autoregressive Factorization	21
10 Training Objective	22
10.1 Teacher Forcing	22
10.2 Negative Log-Likelihood	22
11 Complete Forward Pass	23
11.1 Encoder Path	23
11.2 Decoder Path	23
12 Computational Complexity	25
12.1 Attention Matrix	25
12.2 Feed-Forward Network Complexity	25
13 Core Transformer Configurations	27
13.1 Encoder-only	27
13.2 Decoder-only	27
13.3 Encoder–Decoder	27
14 The Architecture as a Composition of Functions	29
15 Reference Implementation	31

Chapter 1

Transformer Architecture: Foundations

1.1 The Basic Problem

A Transformer is a neural architecture designed to process sequences by allowing elements of the sequence to interact through attention.

Let an input sequence consist of n elements:

$$X = (x_1, x_2, \dots, x_n). \quad (1.1)$$

For language, the elements are tokens. For other modalities, the elements may represent image patches, audio segments, time-series patches, graph nodes, or other structured units.

The Transformer does not generally operate directly on the raw objects x_i . Each input element must first be converted into a vector representation.

Let the model dimension be

$$d_{\text{model}}. \quad (1.2)$$

The Transformer therefore operates on a sequence of vectors

$$H = \begin{bmatrix} h_1^T \\ h_2^T \\ \vdots \\ h_n^T \end{bmatrix} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (1.3)$$

The fundamental operation of the Transformer is to construct a new representation for each position using information from other positions.

For position i ,

$$h'_i = \sum_{j=1}^n \alpha_{ij} v_j, \quad (1.4)$$

where α_{ij} determines how strongly position i uses information from position j .

The Transformer therefore requires a mechanism for computing the coefficients α_{ij} .

That mechanism is attention.

1.2 Sequence Representation

For a language model, suppose the vocabulary contains V possible tokens.

The tokenizer converts an input sequence into integer indices:

$$(t_1, t_2, \dots, t_n), \quad t_i \in \{0, 1, \dots, V-1\}. \quad (1.5)$$

Let the token embedding matrix be

$$E \in \mathbb{R}^{V \times d_{\text{model}}}. \quad (1.6)$$

The embedding corresponding to token t_i is

$$e_i = E[t_i]. \quad (1.7)$$

Consequently, the complete embedded sequence is

$$H_{\text{token}} = \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_n^T \end{bmatrix} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (1.8)$$

At this stage, each token has a vector representation, but the representation must also contain information about position.

1.3 Why Positional Information Is Required

Self-attention by itself does not impose an ordering on the input positions.

Consider two sequences containing the same representations but in different orders:

$$(h_1, h_2, h_3) \quad (1.9)$$

and

$$(h_3, h_1, h_2). \quad (1.10)$$

Without an explicit positional mechanism, attention does not intrinsically encode the distinction between these orderings.

The original Transformer therefore adds positional encodings to the token embeddings.

1.4 Sinusoidal Positional Encoding

For position pos and embedding dimension index i , the original Transformer defines

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (1.11)$$

and

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (1.12)$$

The positional encoding matrix is therefore

$$PE \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (1.13)$$

The input to the first Transformer layer is

$$H^{(0)} = H_{\text{token}} + PE. \quad (1.14)$$

Thus the initial sequence representation is

$$H^{(0)} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (1.15)$$

The positional mechanism is one of the components later modified by Transformer variants.

Chapter 2

The Original Transformer

2.1 Overall Structure

The original Transformer, introduced by Vaswani et al., is an encoder–decoder architecture.

The original paper is:

Attention Is All You Need

The architecture consists of two stacks:

1. an encoder stack;
2. a decoder stack.

The encoder transforms the source sequence into contextual representations.

The decoder uses those representations while generating the target sequence.

The complete structure can be represented as

$$X \rightarrow \text{Embedding} \rightarrow \text{Positional Encoding} \rightarrow \text{Encoder} \rightarrow H_{\text{enc}}, \quad (2.1)$$

followed by

$$H_{\text{enc}}, Y_{<t} \rightarrow \text{Decoder} \rightarrow \text{Output Distribution}. \quad (2.2)$$

2.2 Encoder

Suppose the encoder contains L layers.

The initial representation is

$$H^{(0)} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (2.3)$$

The encoder applies a sequence of transformations:

$$H^{(0)} \rightarrow H^{(1)} \rightarrow H^{(2)} \rightarrow \dots \rightarrow H^{(L)}. \quad (2.4)$$

The final encoder representation is

$$H_{\text{enc}} = H^{(L)}. \quad (2.5)$$

Each encoder layer contains:

1. multi-head self-attention;
2. a position-wise feed-forward network;
3. residual connections;
4. layer normalization.

The attention mechanism is applied first, followed by the feed-forward transformation.

Chapter 3

Self-Attention

3.1 Input to the Attention Layer

Consider the representation entering an attention sublayer:

$$H \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (3.1)$$

The attention mechanism transforms H into queries, keys and values.

Three learned projection matrices are used:

$$W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad (3.2)$$

$$W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad (3.3)$$

and

$$W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}. \quad (3.4)$$

The projections are

$$Q = HW^Q, \quad (3.5)$$

$$K = HW^K, \quad (3.6)$$

and

$$V = HW^V. \quad (3.7)$$

Their dimensions are

$$Q, K \in \mathbb{R}^{n \times d_k}, \quad (3.8)$$

and

$$V \in \mathbb{R}^{n \times d_v}. \quad (3.9)$$

The same input H generates all three matrices in self-attention.

3.2 Queries, Keys and Values

Let

$$q_i \in \mathbb{R}^{d_k} \quad (3.10)$$

be the query associated with position i .

Let

$$k_j \in \mathbb{R}^{d_k} \quad (3.11)$$

be the key associated with position j .

The attention mechanism determines how relevant k_j is to q_i .

The value

$$v_j \in \mathbb{R}^{d_v} \quad (3.12)$$

contains the information that can be transferred from position j to position i .

The distinction is therefore:

$$q_i \text{ asks what position } i \text{ is looking for,} \quad (3.13)$$

$$k_j \text{ describes what position } j \text{ offers,} \quad (3.14)$$

and

$$v_j \text{ contains the information retrieved from position } j. \quad (3.15)$$

3.3 Query–Key Interaction

The compatibility between every query and every key is computed using the matrix product

$$S = QK^T. \quad (3.16)$$

Since

$$Q \in \mathbb{R}^{n \times d_k} \quad (3.17)$$

and

$$K^T \in \mathbb{R}^{d_k \times n}, \quad (3.18)$$

the resulting matrix is

$$S \in \mathbb{R}^{n \times n}. \quad (3.19)$$

Its (i, j) element is

$$S_{ij} = q_i^T k_j. \quad (3.20)$$

Therefore, each row corresponds to one query position and each column corresponds to one key position.

For a fixed position i ,

$$S_{i,:} = [q_i^T k_1, q_i^T k_2, \dots, q_i^T k_n]. \quad (3.21)$$

The model therefore obtains a score describing the relationship between the query at position i and every position in the sequence.

3.4 Scaling

The raw dot products are divided by

$$\sqrt{d_k}. \quad (3.22)$$

Thus,

$$\tilde{S} = \frac{QK^T}{\sqrt{d_k}}. \quad (3.23)$$

The purpose of this scaling is to prevent the magnitude of the dot products from growing excessively as d_k increases.

3.5 Softmax

The scaled scores are converted into attention weights using softmax.

For position i attending to position j ,

$$A_{ij} = \frac{\exp\left(\frac{q_i^T k_j}{\sqrt{d_k}}\right)}{\sum_{r=1}^n \exp\left(\frac{q_i^T k_r}{\sqrt{d_k}}\right)}. \quad (3.24)$$

The resulting attention matrix is

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right), \quad (3.25)$$

where softmax is applied independently across each row.

Therefore,

$$A \in \mathbb{R}^{n \times n}. \quad (3.26)$$

Each row satisfies

$$\sum_{j=1}^n A_{ij} = 1. \quad (3.27)$$

3.6 Value Aggregation

The attention weights are used to combine the value vectors:

$$Z = AV. \quad (3.28)$$

Since

$$A \in \mathbb{R}^{n \times n} \quad (3.29)$$

and

$$V \in \mathbb{R}^{n \times d_v}, \quad (3.30)$$

we obtain

$$Z \in \mathbb{R}^{n \times d_v}. \quad (3.31)$$

For a particular position i ,

$$z_i = \sum_{j=1}^n A_{ij} v_j. \quad (3.32)$$

Therefore, the representation at position i becomes a weighted combination of information from the entire sequence.

3.7 Complete Scaled Dot-Product Attention

Combining the preceding operations gives

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V}. \quad (3.33)$$

The complete computational progression is therefore

$$H \rightarrow (Q, K, V) \rightarrow QK^T \rightarrow \frac{QK^T}{\sqrt{d_k}} \rightarrow \text{softmax} \rightarrow A \rightarrow AV \rightarrow Z. \quad (3.34)$$

This sequence of operations is the fundamental mathematical unit on which the later attention variants are constructed.

Chapter 4

Multi-Head Attention

4.1 Motivation

A single attention operation produces one representation of relationships between positions.

The original Transformer instead performs several attention operations in parallel.

Let the number of heads be h .

For head r , separate projection matrices are used:

$$W_r^Q, \quad W_r^K, \quad W_r^V. \quad (4.1)$$

For each head,

$$Q_r = HW_r^Q, \quad (4.2)$$

$$K_r = HW_r^K, \quad (4.3)$$

and

$$V_r = HW_r^V. \quad (4.4)$$

4.2 Attention in Each Head

Head r computes

$$Z_r = \text{softmax} \left(\frac{Q_r K_r^T}{\sqrt{d_k}} \right) V_r. \quad (4.5)$$

Thus the complete operation is performed independently for each head.

4.3 Concatenation

The outputs of the heads are concatenated:

$$Z_{\text{cat}} = \text{Concat} (Z_1, Z_2, \dots, Z_h). \quad (4.6)$$

A final learned projection is applied:

$$Z_{\text{MHA}} = Z_{\text{cat}} W^O. \quad (4.7)$$

Therefore,

$$\boxed{\text{MHA}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O}. \quad (4.8)$$

Each head has its own projections and can therefore construct a different representation of the relationships in the sequence.

4.4 Dimensions

In the usual formulation,

$$d_k = d_v = \frac{d_{\text{model}}}{h}. \quad (4.9)$$

For each head,

$$Q_r, K_r \in \mathbb{R}^{n \times d_k}, \quad (4.10)$$

and

$$V_r \in \mathbb{R}^{n \times d_v}. \quad (4.11)$$

After concatenating h heads,

$$Z_{\text{cat}} \in \mathbb{R}^{n \times h d_v}. \quad (4.12)$$

The output projection maps this back to the model dimension:

$$W^O \in \mathbb{R}^{h d_v \times d_{\text{model}}}. \quad (4.13)$$

Consequently,

$$Z_{\text{MHA}} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (4.14)$$

This allows the attention sublayer to be inserted into the Transformer block while preserving the representation dimension.

Chapter 5

Residual Connections and Normalization

5.1 Residual Connection

Let

$$H \tag{5.1}$$

be the input to an attention sublayer and let

$$Z_{\text{MHA}} \tag{5.2}$$

be its output.

The residual connection forms

$$R = H + Z_{\text{MHA}}. \tag{5.3}$$

The dimensionality remains

$$R \in \mathbb{R}^{n \times d_{\text{model}}}. \tag{5.4}$$

Residual connections provide a direct computational path from the input of a sublayer to its output.

5.2 Layer Normalization

For a representation

$$x \in \mathbb{R}^{d_{\text{model}}}, \tag{5.5}$$

LayerNorm computes the mean

$$\mu = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} x_i, \tag{5.6}$$

and variance

$$\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} (x_i - \mu)^2. \quad (5.7)$$

The normalized vector is

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}. \quad (5.8)$$

Learned parameters γ and β then produce

$$y_i = \gamma_i \hat{x}_i + \beta_i. \quad (5.9)$$

Layer normalization is applied independently to each token representation.

5.3 Attention Sublayer

The original Transformer can therefore be expressed at this stage as

$$H \rightarrow \text{MHA}(H) \rightarrow H + \text{MHA}(H) \rightarrow \text{LayerNorm}. \quad (5.10)$$

Denoting the resulting representation by H_{attn} ,

$$H_{\text{attn}} = \text{LayerNorm}(H + \text{MHA}(H)). \quad (5.11)$$

Chapter 6

Position-wise Feed-Forward Network

6.1 Input

The output of the attention sublayer,

$$H_{\text{attn}} \in \mathbb{R}^{n \times d_{\text{model}}}, \quad (6.1)$$

is passed through a feed-forward network.

The feed-forward network is applied independently to every position.

For a single token representation x ,

$$x \in \mathbb{R}^{d_{\text{model}}}. \quad (6.2)$$

6.2 First Linear Transformation

Let the hidden dimension of the feed-forward network be d_{ff} .

The first transformation is

$$u = xW_1 + b_1, \quad (6.3)$$

where

$$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad (6.4)$$

and

$$b_1 \in \mathbb{R}^{d_{\text{ff}}}. \quad (6.5)$$

Thus,

$$u \in \mathbb{R}^{d_{\text{ff}}}. \quad (6.6)$$

6.3 Nonlinear Transformation

The original Transformer uses the ReLU activation:

$$\text{ReLU}(u_i) = \max(0, u_i). \quad (6.7)$$

Therefore,

$$g = \text{ReLU}(u). \quad (6.8)$$

6.4 Second Linear Transformation

The second projection is

$$y = gW_2 + b_2, \quad (6.9)$$

where

$$W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad (6.10)$$

and

$$b_2 \in \mathbb{R}^{d_{\text{model}}}. \quad (6.11)$$

Consequently,

$$y \in \mathbb{R}^{d_{\text{model}}}. \quad (6.12)$$

The complete feed-forward transformation is therefore

$$\boxed{\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2}. \quad (6.13)$$

6.5 Application to the Complete Sequence

The FFN is applied independently to each row of H_{attn} .

Thus,

$$\text{FFN}(H_{\text{attn}}) = \begin{bmatrix} \text{FFN}(h_1) \\ \text{FFN}(h_2) \\ \vdots \\ \text{FFN}(h_n) \end{bmatrix}. \quad (6.14)$$

Unlike self-attention, the FFN does not directly mix information between different sequence positions.

Attention performs interaction between positions.

The FFN performs a nonlinear transformation of the representation at each position.

Chapter 7

Complete Encoder Layer

Let the input to an encoder layer be

$$H^{(l-1)} \in \mathbb{R}^{n \times d_{\text{model}}}. \quad (7.1)$$

The self-attention operation produces

$$Z_{\text{attn}} = \text{MHA} \left(H^{(l-1)} \right). \quad (7.2)$$

The residual and normalization operation gives

$$U^{(l)} = \text{LayerNorm} \left(H^{(l-1)} + Z_{\text{attn}} \right). \quad (7.3)$$

The feed-forward network is then applied:

$$Z_{\text{ffn}} = \text{FFN} \left(U^{(l)} \right). \quad (7.4)$$

The second residual and normalization operation gives

$$H^{(l)} = \text{LayerNorm} \left(U^{(l)} + Z_{\text{ffn}} \right). \quad (7.5)$$

Thus the encoder layer can be written as

$$H^{(l-1)} \rightarrow \text{MHA} \rightarrow \text{Residual} \rightarrow \text{LayerNorm} \rightarrow \text{FFN} \rightarrow \text{Residual} \rightarrow \text{LayerNorm} \rightarrow H^{(l)}. \quad (7.6)$$

Repeating this operation L times produces

$$H^{(0)} \rightarrow H^{(1)} \rightarrow \dots \rightarrow H^{(L)}. \quad (7.7)$$

Chapter 8

The Transformer Decoder

8.1 Purpose of the Decoder

The decoder generates the target sequence.

Let the target sequence be

$$Y = (y_1, y_2, \dots, y_m). \quad (8.1)$$

During generation, the prediction at position t must depend only on previously available target tokens:

$$y_1, \dots, y_{t-1}. \quad (8.2)$$

The decoder therefore requires masked self-attention.

8.2 Decoder Input

The previously generated target tokens are converted into embeddings:

$$Y_{<t} = (y_1, \dots, y_{t-1}). \quad (8.3)$$

After token embedding and positional encoding, the decoder representation is

$$D^{(0)} \in \mathbb{R}^{m \times d_{\text{model}}}. \quad (8.4)$$

8.3 Causal Mask

For decoder self-attention, position i must not attend to future position $j > i$.

Define a mask

$$M \in \{0, -\infty\}^{m \times m}, \quad (8.5)$$

with

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (8.6)$$

The masked attention scores are

$$S_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M. \quad (8.7)$$

The attention matrix is therefore

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right). \quad (8.8)$$

Because

$$\exp(-\infty) = 0, \quad (8.9)$$

future positions receive zero attention weight.

8.4 Decoder Self-Attention

The decoder self-attention computation is

$$Q = DW^Q, \quad (8.10)$$

$$K = DW^K, \quad (8.11)$$

$$V = DW^V, \quad (8.12)$$

followed by

$$Z_{\text{self}} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V. \quad (8.13)$$

Thus the decoder representation at position t can depend on positions up to t , but not on future target positions.

8.5 Encoder–Decoder Cross-Attention

The decoder contains a second attention mechanism that connects the decoder to the encoder.

Let

$$H_{\text{enc}} \quad (8.14)$$

be the encoder output.

The queries come from the decoder:

$$Q = DW^Q. \quad (8.15)$$

The keys and values come from the encoder:

$$K = H_{\text{enc}} W^K, \quad (8.16)$$

and

$$V = H_{\text{enc}} W^V. \quad (8.17)$$

Therefore,

$$Q \quad (8.18)$$

is derived from the current decoder representation, while

$$K, V \quad (8.19)$$

are derived from the source sequence representation.

The cross-attention output is

$$Z_{\text{cross}} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (8.20)$$

This operation allows the decoder to retrieve information from the encoded source sequence.

8.6 Decoder Layer

A decoder layer therefore contains three principal transformations:

1. masked self-attention;
2. encoder–decoder cross-attention;
3. position-wise feed-forward network.

The computational progression is

$$D^{(l-1)} \rightarrow \text{Masked Self-Attention} \rightarrow \text{Residual/Normalization} \rightarrow \text{Cross-Attention} \rightarrow \text{Residual/Normalization} \rightarrow \text{FF} \quad (8.21)$$

Chapter 9

Output Generation

9.1 Decoder Representation

After L decoder layers,

$$D^{(L)} \in \mathbb{R}^{m \times d_{\text{model}}}. \quad (9.1)$$

For a vocabulary of size V , a linear projection converts the decoder representation into vocabulary logits:

$$Z = D^{(L)} W_{\text{vocab}} + b_{\text{vocab}}, \quad (9.2)$$

where

$$W_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}} \times V}. \quad (9.3)$$

Therefore,

$$Z \in \mathbb{R}^{m \times V}. \quad (9.4)$$

9.2 Vocabulary Probability Distribution

For position t , the probability of token v is

$$P(y_t = v \mid y_{<t}, X) = \frac{\exp(Z_{t,v})}{\sum_{r=1}^V \exp(Z_{t,r})}. \quad (9.5)$$

Equivalently,

$$P(y_t \mid y_{<t}, X) = \text{softmax}(Z_t). \quad (9.6)$$

The predicted token can then be obtained from this distribution.

9.3 Autoregressive Factorization

The complete target sequence probability is factorized as

$$P(Y | X) = \prod_{t=1}^m P(y_t | y_1, \dots, y_{t-1}, X). \quad (9.7)$$

The decoder therefore generates the target sequence one position at a time. At generation step t ,

$$(y_1, \dots, y_{t-1}) \rightarrow \text{Decoder} \rightarrow P(y_t | y_{<t}, X). \quad (9.8)$$

The newly generated token is then incorporated into the next decoder input.

Chapter 10

Training Objective

10.1 Teacher Forcing

During training, the decoder is supplied with the target sequence shifted by one position.

For example,

$$(y_1, y_2, \dots, y_m) \tag{10.1}$$

is used to predict

$$(y_2, y_3, \dots, y_{m+1}), \tag{10.2}$$

with appropriate start and end tokens.

This allows the model to compute predictions for many positions in parallel during training while the causal mask prevents information from future target positions from being used.

10.2 Negative Log-Likelihood

The training objective is the negative log-likelihood:

$$\mathcal{L} = - \sum_{t=1}^m \log P_{\theta}(y_t \mid y_{<t}, X). \tag{10.3}$$

Equivalently, the objective is commonly expressed as the sum of cross-entropy losses over target positions.

Chapter 11

Complete Forward Pass

The complete original Transformer can now be followed from raw input to output.

11.1 Encoder Path

The source sequence

$$X = (x_1, \dots, x_n) \tag{11.1}$$

is first converted into token indices.

Then,

$$X \rightarrow \text{Tokenization} \rightarrow (t_1, \dots, t_n). \tag{11.2}$$

The token indices are embedded:

$$(t_1, \dots, t_n) \rightarrow H_{\text{token}}. \tag{11.3}$$

Positional information is added:

$$H_{\text{token}} \rightarrow H^{(0)} = H_{\text{token}} + PE. \tag{11.4}$$

The representation then passes through the encoder stack:

$$H^{(0)} \rightarrow H^{(1)} \rightarrow \dots \rightarrow H^{(L)}. \tag{11.5}$$

The final representation is

$$H_{\text{enc}} = H^{(L)}. \tag{11.6}$$

11.2 Decoder Path

The target prefix

$$Y_{<t} = (y_1, \dots, y_{t-1}) \tag{11.7}$$

is embedded and positionally represented:

$$Y_{<t} \rightarrow D^{(0)}. \quad (11.8)$$

The first decoder operation is masked self-attention:

$$D^{(0)} \rightarrow Q, K, V \rightarrow \frac{QK^T}{\sqrt{d_k}} + M \rightarrow \text{softmax} \rightarrow Z_{\text{self}}. \quad (11.9)$$

The resulting representation interacts with the encoder output through cross-attention:

$$D \rightarrow Q_D, \quad (11.10)$$

while

$$H_{\text{enc}} \rightarrow K_{\text{enc}}, V_{\text{enc}}. \quad (11.11)$$

Then,

$$Q_D, K_{\text{enc}}, V_{\text{enc}} \rightarrow Z_{\text{cross}}. \quad (11.12)$$

The result is transformed by the feed-forward network.

After all decoder layers,

$$D^{(0)} \rightarrow D^{(1)} \rightarrow \dots \rightarrow D^{(L)}. \quad (11.13)$$

Finally,

$$D^{(L)} \rightarrow \text{Linear Projection} \rightarrow \text{Vocabulary Logits} \rightarrow \text{softmax} \rightarrow P(y_t \mid y_{<t}, X). \quad (11.14)$$

Chapter 12

Computational Complexity

12.1 Attention Matrix

For

$$Q, K \in \mathbb{R}^{n \times d_k}, \quad (12.1)$$

the product

$$QK^T \quad (12.2)$$

produces

$$n \times n \quad (12.3)$$

attention scores.

The computation therefore scales approximately as

$$O(n^2 d_k). \quad (12.4)$$

The attention matrix itself requires

$$O(n^2) \quad (12.5)$$

memory.

This quadratic dependence on sequence length is one of the principal motivations for later Transformer variants.

12.2 Feed-Forward Network Complexity

For each token,

$$x \rightarrow xW_1 \rightarrow \text{ReLU} \rightarrow W_2 \quad (12.6)$$

requires approximately

$$O(d_{\text{model}}d_{\text{ff}}) \tag{12.7}$$

operations.

Across n positions,

$$O(nd_{\text{model}}d_{\text{ff}}). \tag{12.8}$$

Thus the principal sequence-length dependence of standard self-attention is quadratic, whereas the position-wise FFN scales linearly with sequence length.

Chapter 13

Core Transformer Configurations

The original Transformer establishes three architectural configurations that became the basis for subsequent Transformer families.

13.1 Encoder-only

An encoder-only architecture retains the encoder stack and removes the autoregressive decoder.

The computational path is

$$X \rightarrow \text{Embedding} \rightarrow \text{Position Information} \rightarrow \text{Encoder Stack} \rightarrow H_{\text{enc}} \rightarrow \text{Task Head}. \quad (13.1)$$

Because the self-attention is not required to be causal, a token can generally attend to other positions in the input sequence.

This configuration is particularly suitable for obtaining contextual representations.

Examples include BERT-family architectures.

13.2 Decoder-only

A decoder-only architecture removes the encoder and retains causal Transformer blocks.

The computational path becomes

$$X \rightarrow \text{Embedding} \rightarrow \text{Positional Mechanism} \rightarrow \text{Causal Self-Attention} \rightarrow \text{FFN} \rightarrow \dots \rightarrow \text{Vocabulary Distribution}. \quad (13.2)$$

The probability factorization is

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_{<t}). \quad (13.3)$$

GPT-family language models are based on this general configuration.

13.3 Encoder–Decoder

The encoder–decoder configuration retains both stacks.

The source sequence is transformed into

$$X \rightarrow \text{Encoder} \rightarrow H_{\text{enc}}. \quad (13.4)$$

The target prefix is processed by the decoder:

$$Y_{<t} \rightarrow \text{Masked Self-Attention}. \quad (13.5)$$

The decoder then accesses the encoder representation:

$$(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) \rightarrow \text{Cross-Attention}. \quad (13.6)$$

Finally,

$$D^{(L)} \rightarrow \text{Vocabulary Projection} \rightarrow P(y_t \mid y_{<t}, X). \quad (13.7)$$

This configuration is naturally suited to sequence-to-sequence tasks.

Chapter 14

The Architecture as a Composition of Functions

The Transformer can be understood as a composition of parameterized functions.

Let the input representation be

$$H^{(0)}. \quad (14.1)$$

A generic Transformer block can be written as

$$H^{(l)} = F_l \left(H^{(l-1)} \right). \quad (14.2)$$

For an encoder layer,

$$F_l = F_{\text{norm}} \circ F_{\text{res}} \circ F_{\text{FFN}} \circ F_{\text{norm}} \circ F_{\text{res}} \circ F_{\text{MHA}}. \quad (14.3)$$

More explicitly,

$$H^{(l-1)} \rightarrow \text{MHA} \rightarrow \text{Add} \rightarrow \text{LayerNorm} \rightarrow \text{FFN} \rightarrow \text{Add} \rightarrow \text{LayerNorm} \rightarrow H^{(l)}. \quad (14.4)$$

The full encoder is therefore

$$H_{\text{enc}} = F_L \circ F_{L-1} \circ \dots \circ F_1 \left(H^{(0)} \right). \quad (14.5)$$

This functional view is useful because Transformer variants can be classified according to which function is replaced.

For example,

$$F_{\text{MHA}} \rightarrow F_{\text{sparse attention}} \quad (14.6)$$

produces an attention-efficiency variant.

Similarly,

$$F_{\text{position}} \rightarrow F_{\text{RoPE}} \quad (14.7)$$

changes the positional mechanism.

A change to the input representation gives a modality-specific Transformer, while a change to the feed-forward function gives a different Transformer block parameterization.

This provides the basis for the systematic treatment of Transformer variants in the subsequent parts.

Chapter 15

Reference Implementation

The original Transformer paper provides the architectural definition from which the subsequent variants developed.

The paper is available at:

<https://arxiv.org/abs/1706.03762>

A widely used implementation framework for Transformer architectures is the Hugging Face Transformers library:

<https://github.com/huggingface/transformers>

The implementation should be understood only after the mathematical computation has been established:

$$\text{Input} \rightarrow \text{Representation} \rightarrow Q, K, V \rightarrow \text{Attention} \rightarrow \text{Residual/Normalization} \rightarrow \text{FFN} \rightarrow \text{Repeated Blocks} \rightarrow \text{Output} \quad (15.1)$$

The later Transformer variants modify specific stages of this pipeline.